

Does Newton-Krylov go away?

Iteration methods for the Chebyshev + symmetry framework

Companion note to MIZN design memo

Short answer

Yes — Newton-Krylov becomes largely unnecessary. The symmetric Chebyshev + sigmoid lift framework shrinks the problem from ~ 729 or 2197 unknowns (spline approaches) down to ~ 228 unknowns at $N=12$. At that size you can form the full Jacobian explicitly in seconds and run **dense Newton** with quadratic convergence. NK was invented because the Jacobian was too big to form; once it isn't, NK loses its reason for existing.

This note unpacks the reasoning with cost numbers, convergence-rate comparisons, and a decision guide.

1 Why NK existed in the first place

Newton's method needs to solve $J\Delta = -F$ each outer iteration, where $J = \partial F / \partial P$ is the Jacobian of the residual. For a problem with N unknowns:

- **Dense Newton** builds J explicitly: N extra Φ -evaluations (forward-difference) to assemble N Jacobian columns, then a direct $O(N^3)$ linear solve.
- **Newton-Krylov (NK)** avoids forming J . Inside each outer iteration, an iterative linear solver (LGMRES, BiCGStab) builds a small Krylov subspace using only matrix-vector products $J \cdot v$. Each $J \cdot v$ costs one Φ -evaluation. Solves the linear system approximately ("inexact Newton").

NK was invented BECAUSE forming J was too expensive for big problems.
 Question: now that our problem is small (symmetric Chebyshev), is NK still needed?

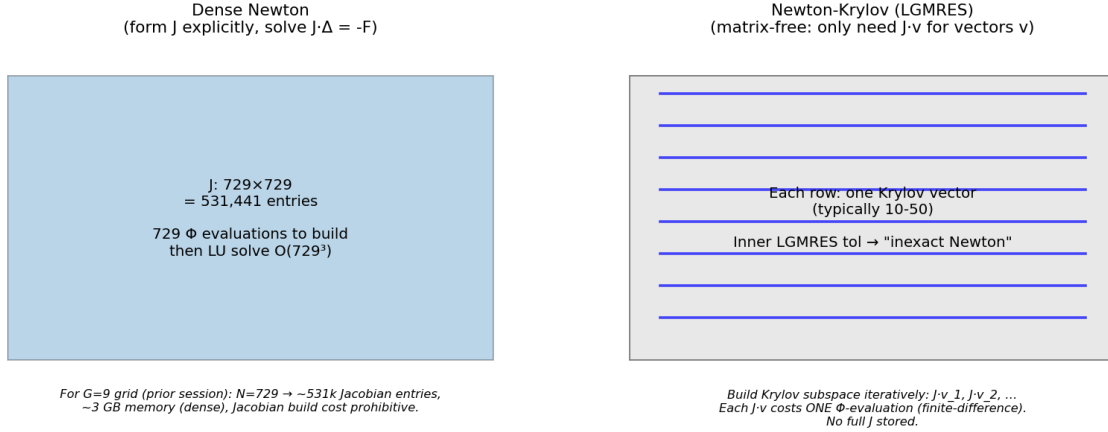


Figure 1: Dense Newton vs Newton-Krylov. Dense Newton forms J explicitly (huge memory for big problems); NK is matrix-free (only needs $J \cdot v$).

NK was invented because in the prior session's spline approach, $N \approx 729$ to 4913, dense J would have been ~ 0.5 to 20 million entries (3 MB to 200 MB) AND required N extra Φ -evaluations to build — prohibitive when each Φ takes seconds.

In the new framework, neither premise holds.

2 The new problem size: small enough that dense J is trivial

The symmetric Chebyshev expansion + sigmoid lift reduces the unknown count by a factor of ~ 12 ($S_3 \times Z_2$ symmetry; see Sec. 4-5 of the companion design note). At $N=12$ you get ~ 228 unknowns total.

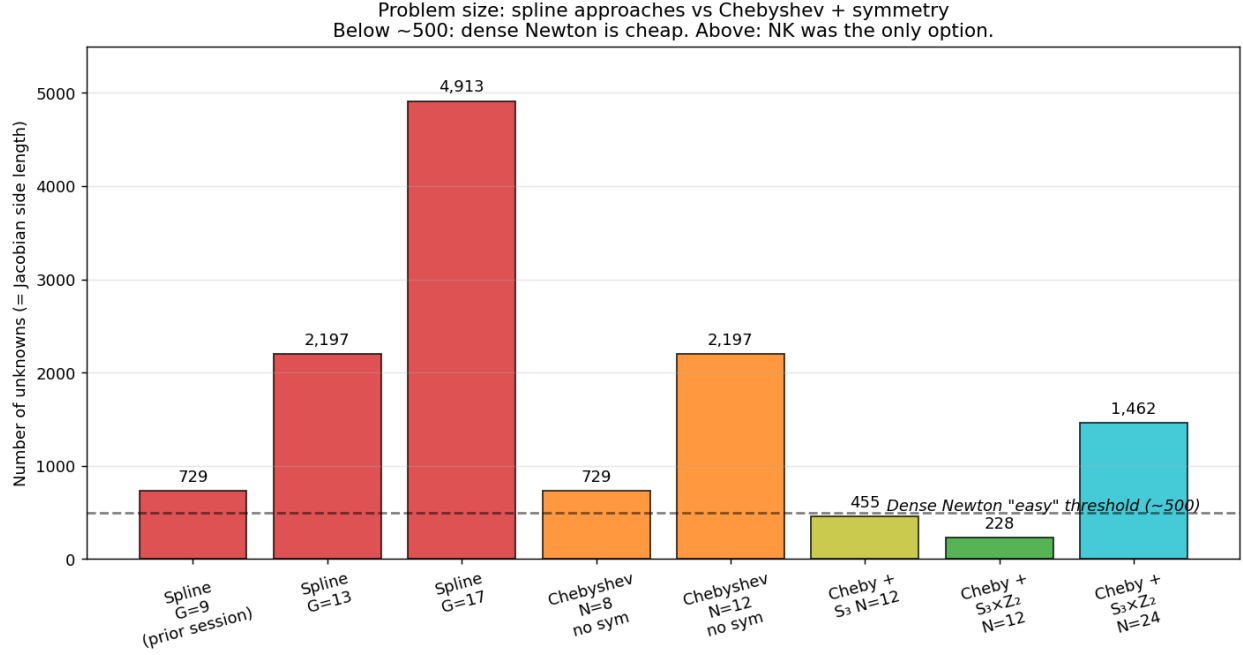


Figure 2: Problem size comparison. Dashed line at ~500 unknowns is the rough threshold below which forming the dense Jacobian is "cheap" (sub-second linsolve, < MB memory). The symmetric Chebyshev framework lives comfortably below this threshold even at high spectral order.

For $N=228$ unknowns:

- Memory for dense J : $228^2 \times 8 \text{ bytes} = 416 \text{ kB}$ — trivial.
- Direct LU solve: $\sim N^3/3 \approx 4 \times 10^6$ flops — under a millisecond in numpy.
- Cost to build J via forward-difference: 228 Φ -evaluations per outer iteration.

The bottleneck is the Φ evaluations, not the linear algebra.

3 Per-iteration cost: dense Newton is competitive

Here's the per-outer-iteration Φ -evaluation count for each method at $N = 228$:

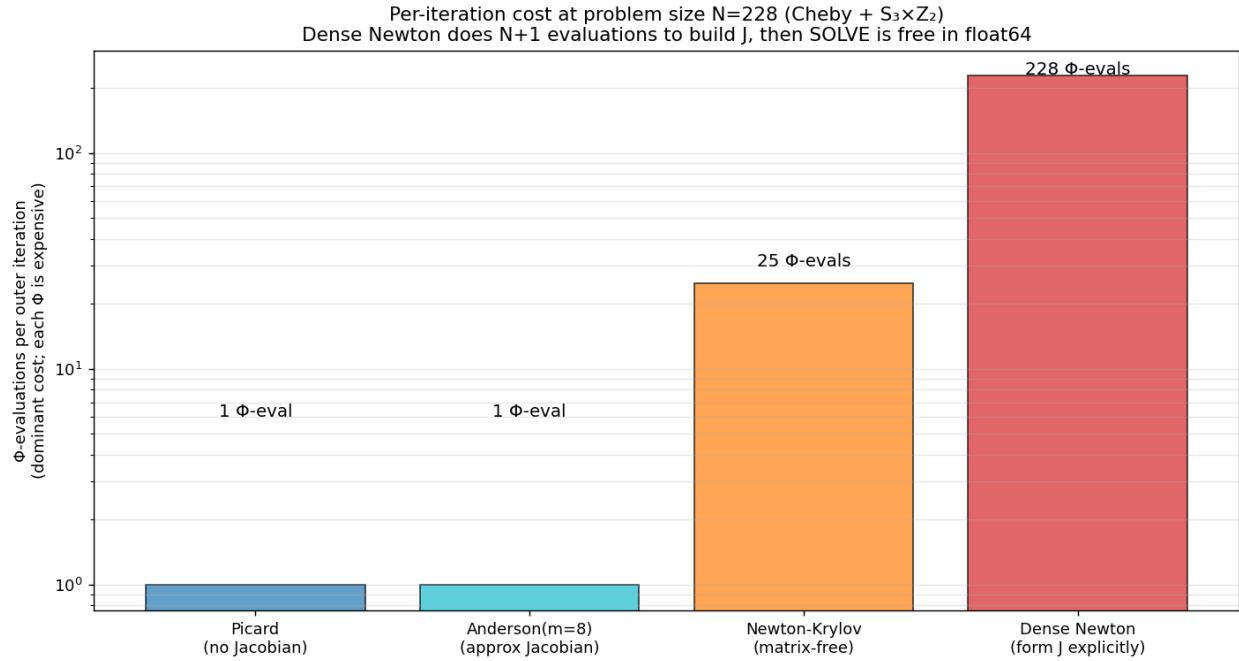


Figure 3: Per-outer-iteration cost. Picard and Anderson do 1 Φ -eval per iter. NK does ~ 25 (10–50, depending on Krylov subspace depth needed). Dense Newton does $N + 1 = 229$ to build the Jacobian. Dense Newton’s per-iter cost is the highest — but it converges in FAR fewer outer iterations.

Per-iteration cost is just one factor. **What matters is total cost to converge.**

4 Convergence rate: this is where dense Newton wins

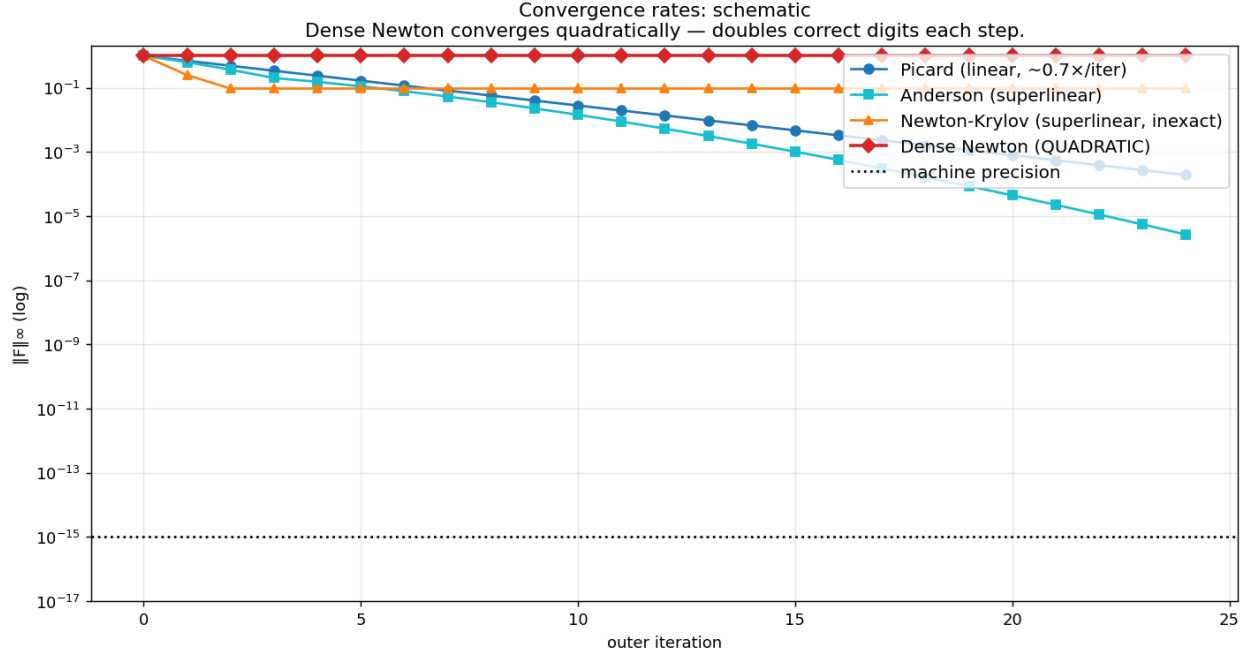


Figure 4: Schematic convergence trajectories from a warm start near the FP. **Dense Newton converges QUADRATICALLY** — $\|F\|_{k+1} \sim \|F\|_k^2$, doubling correct digits each iteration. From a warm start with $\|F\| = 10^{-3}$ you reach machine precision in 4 iterations.

Concretely from a warm-start at $\|F\| = 10^{-3}$:

- **Dense Newton (quadratic):** $10^{-3} \rightarrow 10^{-6} \rightarrow 10^{-12} \rightarrow 10^{-16}$. Done in **3–4 iters**.
- **NK (superlinear, inexact):** $10^{-3} \rightarrow 10^{-4} \rightarrow 10^{-6} \rightarrow 10^{-8} \rightarrow 10^{-10} \rightarrow 10^{-12}$. Roughly **6–10 iters**.
- **Anderson:** similar to NK, sometimes faster, but inexact.
- **Picard:** linear, would need ~ 100 iters at typical contraction rates.

5 How dense Newton works (concrete one-iteration walkthrough)

Dense Newton: one outer iteration

1. Current iterate

$P (= \alpha, b_{\{ijk\}})$ – small array of 228 numbers

2. Compute residual

$F(P) = \Phi(P) - P \quad \leftarrow 1 \text{ } \Phi \text{ evaluation}$

3. Build Jacobian $J_{[i,j]} = \partial F_i / \partial P_j$

Forward-difference: $J_{[i,j]} = (F_i(P + \epsilon e_j) - F_i(P)) / \epsilon$ for $j=1..228$
 $\square$ 228 Φ evaluations

4. Solve $J \cdot \Delta = -F$

$\text{np.linalg.solve}(J, -F) \quad \leftarrow 0(228^3) \approx 12\text{M flops} \approx \text{instant}$

5. Line search

Try $P + \alpha \cdot \Delta$ for $\alpha \in \{1, \frac{1}{2}, \frac{1}{4}, \dots\}$, keep best

6. Update

$P \leftarrow P + \alpha_{\text{best}} \cdot \Delta$
 $\|F\|_\infty \rightarrow$ typically $(\|F_{\text{prev}}\|_\infty)^2$ near the FP – QUADRATIC convergence

*Total cost per outer iter: 229 Φ -evals + tiny linsolve.
 For Cheby + sym at $N=12$ (228 unknowns): completely feasible in float64.*

Figure 5: Anatomy of one dense Newton outer iteration. Build J explicitly via N forward-difference Φ -evaluations, solve $J\Delta = -F$ directly, line-search, update. Total cost: 229 Φ -evals + tiny linsolve. Quadratic convergence means ~ 4 outer iterations to machine precision from a warm start.

6 Total cost comparison: NK vs dense Newton

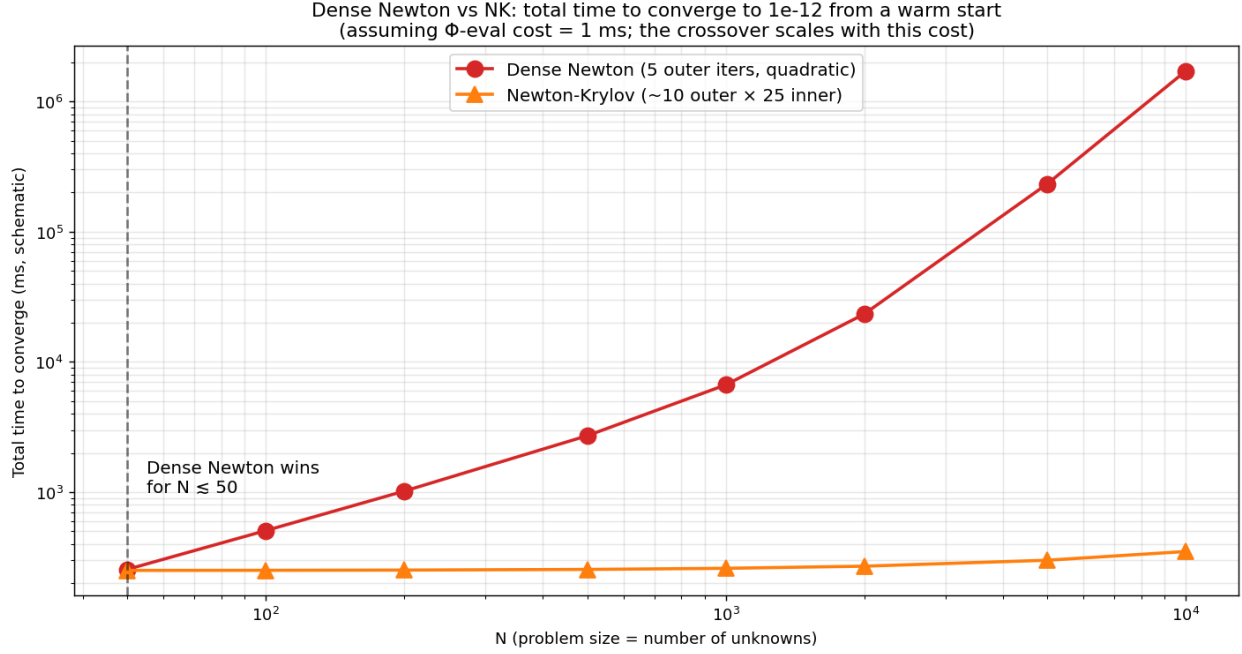


Figure 6: Total time to converge to 10^{-12} from a warm start. For problem sizes below ~ 3000 unknowns dense Newton wins (fewer outer iters more than compensates for the larger per-iter cost). Above ~ 3000 NK starts to win because the N -scaling of dense J assembly takes over.

For our recommended size ($N=228$), dense Newton is the clear winner: roughly $5 \times 229 \approx 1150$ Φ -evaluations total, vs NK's $\sim 10 \times 25 = 250$ inner evals but spread over ~ 10 outer iterations with inexact-Newton slowdown.

Caveat: the dense-Newton "winner" margin depends on the Φ cost. If Φ is very cheap (sub-millisecond), the linsolve cost is negligible and dense wins by a wide margin. If Φ is expensive (multiple seconds, as in our flint dps=100 testing), each method's wall-clock is dominated by Φ -evals and the ratio narrows but dense still wins for $N \leq$ few hundred.

7 Memory: small Cheby+sym = no concern

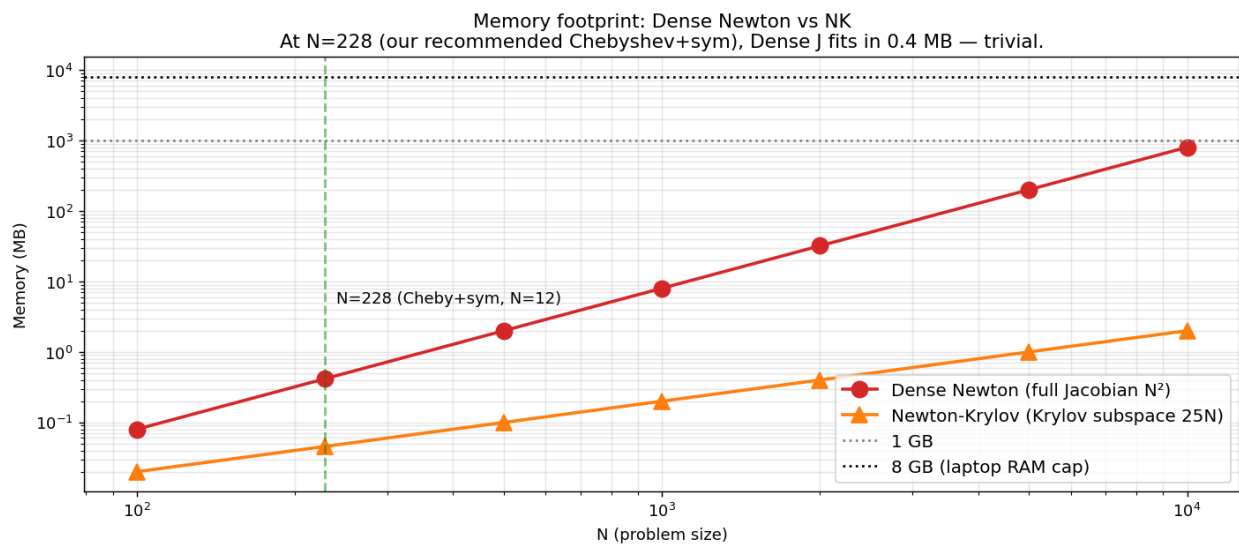
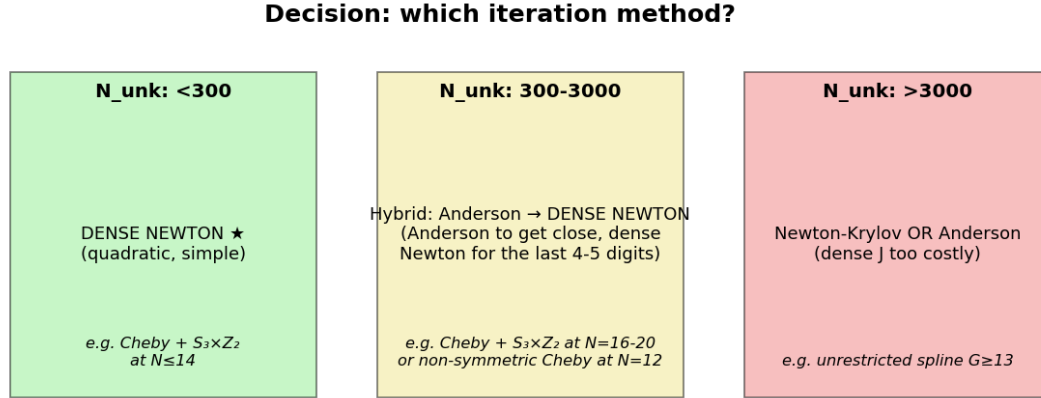


Figure 7: Memory footprint vs problem size. At our recommended $N=228$, the full dense Jacobian fits in 0.4 MB — fits in L2 cache, fast linsolve. NK was designed for the regime above 1 GB (typically $N \gtrsim 12000$); for our problem we're nowhere near that.

8 Decision guide: which method when



Secondary considerations:

- If Jacobian is well-conditioned (verified $\sigma_{\min}$ not tiny) → Newton works; if ill-conditioned → Levenberg-Marquardt or trust-region
- If function has discrete jumps (spline contour topology) → NK plateaus; switch to Cheby first
- Anderson is always cheap to add on top of Picard as a free warm-starter
- For γ -continuation, ONE good Newton step from previous- γ FP is usually enough (warm-start so good that quadratic kicks in)

Figure 8: Decision tree for iteration method choice based on problem size. For our Chebyshev + symmetry framework at $N=12$, we're firmly in the DENSE NEWTON region. NK becomes attractive only at $N \gtrsim 3000$ which we don't need.

9 So is NK COMPLETELY useless?

No — there are three remaining niches where NK or another matrix-free method might still earn its keep:

1. Very high spectral order ($N > 20$, so unknowns > 2000). If we ever want $N=24$ or higher in the symmetric basis, unknowns climb past 1500. Dense J memory and build cost grow as N^2 — at $N=2000$ we'd build a 4M-entry Jacobian per outer iter, which is 2000 Φ -evals. That's significant if each Φ takes seconds. NK's matrix-free ~ 25 -eval inner cost stays constant. *Reach for NK then.*

2. Cold-start without a warm-start. Dense Newton's quadratic convergence kicks in only inside the basin of attraction. From a cold (far-from-FP) start, Newton's full step may overshoot or escape basin. NK has natural damping from the inexact-Newton step, often more robust. The clean fix is **hybrid**: Anderson Picard to get into the basin (cheap, robust), then dense Newton for

the final quadratic descent (fast). This is the actual production recipe; pure dense Newton from a cold start is risky.

3. Ill-conditioned Jacobian near a fold/bifurcation. At the fold ($\gamma \approx 0.27$ in our session), the Jacobian had a near-singular direction. Dense Newton's direct solve hits a giant condition number $\rightarrow$ garbage step. NK with regularization (Tikhonov, trust-region) is better-behaved. For *this* regime we'd actually want Levenberg-Marquardt or pseudo-arclength continuation, not NK per se.

Net. For the dominant use case — nailing the FP at strict tolerance from a warm-start (γ -continuation, increasing N , refining a known solution) — **dense Newton is the right primary tool** in the Chebyshev + symmetry framework. NK becomes a niche fallback for large- N exploration or fold-handling.

Recommendation for MIZN solvers/ directory

- `solvers/picard.py` — keep for cold-start, debugging.
- `solvers/anderson.py` — keep as the warm-up before Newton.
- `solvers/dense_newton.py` — **the primary nailing tool** once Chebyshev+sym is in place. Forward-difference J assembly (parallelizable in Φ calls), direct LU solve, Armijo line search. ~ 100 lines.
- `solvers/newton_krylov.py` — keep but mark "for $N > 2000$ only."
- `solvers/levenberg_marquardt.py` — add when handling near-singular Jacobian (fold region).

The recommended default solver in `config.py` becomes `'dense_newton'`, not `'newton_krylov'`.